

Style guide: Ruby

A [rubocop file](#) is provided in QPixel and rubocop is included in the QPixel bundle. Use `bundle exec rubocop` for Ruby style checking; you can also use `bundle exec rubocop -Da` for auto-correction.

In general, we follow “the” [Ruby Style Guide](#), with a few exceptions. The most important rules and those exceptions are detailed here. Exceptions and departures from the Style Guide are highlighted in **purple**.

Most editors can (and should!) be configured to automatically apply many of these code style rules for you, and to check your code as you write.

You should also read about [when it's appropriate to break these rules](#).

Indentation

Use spaces for indentation; do not use tab stops. Use two spaces per indentation level.

```
1 def my_method
2   'something'
3 end
```

Maximum Line Length

Limit lines to **120** characters. This rule is **not enforced for lines that must contain long URLs**, as splitting them generally introduces more chance for error and makes the code less readable. Use a linter exception to avoid generating rubocop errors for these.

Spaces & Braces

No space after `(` or `[` or before `)` or `]`. Use spaces around `{` and `}`, **including for hash literals**.

Spaces are **not** required around `{` and `}` when used for string interpolation.

```
1 # Not allowed:
2 some( arg )
3 [ 1, 2, 3 ]
4 ary.each{|e| puts e}
5 {a: 1, b: 2, c: 3}
6 "#{ interpolated }"
7
8 # Use instead:
9 some(arg)
10 [1, 2, 3]
11 ary.each { |e| puts e }
12 { a: 1, b: 2, c: 3 }
13 "#{interpolated}"
```

Line Continuation

Line continuation with `\` is only permitted for string literals. Remember that long URLs should **not** be split and are exempt from maximum line lengths.

Multi-line Method Chains

Use the leading `.` style and align the continuation with the initial method call:

```
1 # Not allowed:
2 one.two.three.
3   four
4
5 # Use instead:
6 one.two.three
7   .four
```

Nesting and Including

Use explicit nesting to avoid lexical scope issues:

```
1 # Not allowed:
2 class Something::SubSomething
3   # body omitted
4 end
5
6 # Use instead:
7 module Something
8   class SubSomething
9     # body omitted
10  end
11 end
```

When including modules in classes, do not use a leading `::` unless necessary to resolve correctly:

```
1 # Not allowed:
2 class User
3   include ::Identity
4 end
5
6 # Use instead - unless the current module has a separate Identity
7 # definition
8 # and you want to specify the global one:
9 class User
10  include Identity
11 end
```